Execution-Validated Pseudocode for MATLAB: A Dataset, and Why Compressed Code Prefixes Fail to Condition a Language Model

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—Teaching a language model to explain code requires thousands of code–description pairs, and at that scale the descriptions are written by another language model. This raises a question that existing corpora leave open: can those descriptions be trusted? A description can read well and still be wrong about what the code does. We answer the question with execution. In our pipeline, a description of a MATLAB function enters the dataset only if a second model—which never sees the original code—can rebuild working code from the description alone, and that rebuilt code produces the same outputs as the original when both run in Octave on the same inputs. If the description was incomplete or wrong, the rebuilt program behaves differently and the pair is rejected. One static check keeps this test honest: functions that call helpers missing from the corpus are rejected up front, because they cannot run as written and would pass the test only by accident. From 67,662 corpus functions, 5,296 validated pairs survive. We then used the dataset to test an idea we ourselves believed in: that a compressed vector representation of a program—a syntax-tree summary vector, ViT-style patch embeddings, or both—could replace the code text as input to a decoder. It cannot. A fine-tuned text-only decoder reaches 0.416 ROUGE-L; all three vector variants collapse to 0.17–0.18 and write nearly the same description for every input. Tracing the failure leads to a simple cause: 2.2B decoder parameters were trainable while the code arrived as at most six vectors, so memorizing typical pseudocode was easier than learning to read the vectors. We release the dataset, the pipeline, and the analysis.

Index Terms—Dataset Construction, Execution-Based Validation, Large Language Model, Code Comprehension, Abstract Syntax Tree, Pseudocode Generation

I. Introduction

Developers spend most of their time reading code, not writing it. They study unfamiliar systems, trace legacy behaviour, debug, and document before they change anything [?], [?], [?], [?]. Large Language Models (LLMs) now automate parts of this reading: they summarize, explain, and translate code into plain language [?]. We set out to build such a model for MATLAB—a language that is everywhere in control engineering and scientific computing, yet rare in LLM training data [?].

That goal immediately created a data problem. Training needs thousands of MATLAB functions, each paired with a description of what it does. No one hand-writes thousands of descriptions, so we did what the field does: we had an LLM write them. But this only moves the problem. A generated description can be fluent and still wrong—it can skip a branch, invent a step, or misread an operator. Train on wrong descriptions, and the model learns to produce wrong descriptions. Existing corpora face the same issue and screen their annotations by word overlap with references, or by asking another LLM to judge them. Both check how a description reads. Neither checks whether it is true.

So we asked: how do you verify a description without reading it? The answer we arrived at is the core of this paper. If a description really captures what a program does, then someone given only the description should be able to rebuild a program that behaves the same way. That sentence turns directly into a test (Fig. 1). One model describes the code. A second model, which never sees the original, writes new code from the description alone. Both programs run in GNU Octave on the same inputs. If the outputs match, the description passed; if not, it is rejected. Every one of the 5,296 pairs in our dataset carries the full evidence of its own test: original code, description, rebuilt code, test inputs, and both matching outputs.

Building this pipeline taught us that the test itself has a trap. Our corpus stores one function per file, and many functions call helpers that live elsewhere in their original project. Such a function cannot run as written—but it can still pass the test, for the wrong reason. The model that generates test inputs may quietly invent the missing helper, or the inputs may never reach the line that calls it. Either way the test says “pass” and proves nothing. The fix is to ask before running: a static check extracts every name the code calls and asks Octave whether each one exists. Any function with an unresolvable call is rejected before a single LLM call is spent on it. This one check removes 26.9% of the corpus—and every removed item would otherwise have entered the test with a meaningless result (Section III).

With the dataset in hand, we turned to the question that motivated us in the first place. Code is a tree: functions

contain branches, branches contain loops, loops contain statements. A tokenizer flattens all of it into a string of subwords. We believed that handing the decoder the tree explicitly—as compact learned vectors—would help it understand the code, and might even replace the text entirely. So we trained four variants on the validated data: a plain fine-tuned text decoder, and three variants that swap the code text for compressed vectors (one syntax-tree summary vector, ViT-style patch embeddings, or both).

The result went against our expectation, and the second half of this paper is the story of finding out why. The text decoder reached 0.416 ROUGE-L. All three vector variants collapsed to 0.17–0.18. Looking at their actual outputs revealed something stranger than low scores: the vector models write nearly the same description no matter which program they see. They never name the function they describe; the text model names it 80% of the time. Following that clue led to a simple root cause: our stage-2 training left 2.2B decoder parameters trainable while the program arrived as at most six vectors. The model had two ways to reduce its loss—learn to decode six opaque vectors, or memorize what MATLAB pseudocode typically says—and it took the cheaper one. Section VII walks through the evidence and the three further design flaws that compounded the shortcut.

Our contributions are:

- A dataset. 5,296 MATLAB-to-pseudocode pairs, each validated by rebuilding and re-running the code, with all artefacts included. Dataset and pipeline are public.[1]
- A validation pipeline that guards itself. The self-containedness check stops broken functions from passing the test by accident, and we report exactly where all 67,662 candidates went.
- A negative result, explained. Three compressed-prefix designs collapse into input-independent output. We trace the causes and give three cheap probes that expose this failure in any similar system.

The paper follows three questions in order:

RQ1. Can round-trip execution validate LLM-written pseudocode at corpus scale, and what does it filter out?

RQ2. How well does plain decoder fine-tuning perform on the validated data?

RQ3. Can a handful of learned vectors replace the code text as decoder input?

II. Related Work

Code comprehension. Research on understanding code spans static analysis, AST-based methods, statistical models, and LLMs [?], [?], [?], [?]. Comprehension is measured in many ways, from task time to overlap metrics [?], [?], [?]. Like much recent work, we use pseudocode generation as a measurable proxy for comprehension. Our focus, however, is one step earlier: whether the training data itself can be trusted.

[1]https://huggingface.co/datasets/philip120/sc-matlab-validated

Round-trip acceptance test
1. Describe: code $C \rightarrow$ description S.
2. Rebuild: $S \rightarrow$ code C', by a model that never sees C.
3. Run C and C' in Octave on the same inputs.
4. Keep the pair only if the normalized outputs match.

Fig. 1. A description is accepted only if it is complete enough to rebuild a program with the same behaviour.

Validating generated annotations. Existing code–text corpora screen their annotations with heuristic filters, word-overlap scores, or LLM judges [?]. Execution has been used before—but to check generated code against tests [?], [?]. We flip the direction and use execution to check generated text: the description is the thing under test, and rebuilt code is the measuring instrument. We know of no pseudocode corpus that accepts annotations this way, or that guards the test against unresolvable dependencies.

Structure-aware code models. Many models inject syntax trees into neural networks for code [?], [?], [?], [?], [?], often on top of pre-trained encoders such as CodeBERT or CodeT5 [?]. Patch-style aggregation of embeddings copies the Vision Transformer recipe; for code it is largely untested [?]. Our study stress-tests the most compressed corner of this design space—and documents how it fails.

Soft prefixes and frozen decoders. Prefix tuning and prompt tuning steer a language model with learned vectors [?], [?]. Multimodal systems such as Frozen [?] and BLIP-2 [?] connect an encoder to a frozen LLM through a trained projector. They freeze the decoder on purpose: it forces the training signal through the projector. Our negative result is a demonstration of what happens when that rule is broken.

III. The Dataset

A. The Problem: Descriptions We Could Not Trust

Each MATLAB function needs a step-by-step English description. Ours are written by an LLM (gemini-flash-lite, temperature 0.2). The prompt asks for a description precise enough that "someone could reimplement the code from the pseudocode alone and get identical outputs"—every computed value, every loop bound, every printed message. This gives us descriptions at scale, but no reason to believe any single one of them. So the rest of this section is about earning that belief.

B. The Idea: If It Is True, You Can Rebuild From It

The description's own quality bar suggests its own test. If a description really is precise enough to reimplement the code, then let a model try exactly that—and check the result by running it.

Given code C, we generate a description S. A second, independent generation call receives only S and writes

code C'. If S is faithful, C' should behave like C. We check behaviour by execution: both programs run in Octave, and S is accepted only when their outputs agree.

Two practical questions follow, each with a simple answer. What inputs do the programs run on? A bare function produces no output, so each function runs inside a small generated test script (a harness) that fixes the random seed, builds small numeric inputs, calls the function, and prints every return value. The same harness runs both C and C', so both see identical inputs. The harness prompt has one hard rule—it may only build inputs and make the call, never define or stub a helper—and the next subsection explains why that rule matters. When do outputs "agree"? After normalization: whitespace is cleaned and long floating-point values are rounded to four decimals, so formatting noise cannot cause false rejections.

C. The Trap: Tests That Pass for the Wrong Reason

While building the pipeline we found a failure mode that would have quietly poisoned the dataset. The corpus stores one function per file. A function that calls a helper from its original project cannot run alone—and yet it can still pass the round trip:

- the harness generator, asked for inputs, may silently write the missing helper itself, so the function is tested against invented code; or
- the generated inputs may never reach the branch that calls the helper, so the missing code is never exercised at all.

In both cases the test says "pass" while proving nothing. Left open, this loophole would have filled the dataset with pairs whose validation was hollow—and nothing downstream would ever have noticed.

The fix is to ask before running. From each candidate we extract every name used as a function call, dropping names that are clearly not calls: keywords, local functions, assigned variables, parameters, loop variables, and struct fields. (String literals are blanked first; in MATLAB a bare ' can mean quote or transpose, and unstripped strings look like calls.) Every remaining name goes to Octave's exist: does it resolve to a builtin, a package, or a defined function? One unresolvable name rejects the candidate—before any execution or LLM call is spent on it. Results are cached, so the whole corpus costs only a few interpreter calls.

The scale of the catch surprised us: 18,174 items—26.9% of the whole corpus, more than three rejected for every pair we finally accept. Every one of them had already passed the earlier filters and would otherwise have entered the round trip with a meaningless result (Table I).

D. The Funnel: Where 67,662 Functions Went

The full pipeline, in order: (1) strip comments; (2) quality bounds: 5–100 lines, at most 4,000 characters, at least 50% unique lines; (3) reject file I/O, GUI, and toolbox calls by pattern (fopen, imread, plot, system, eval, …); (4) the self-containedness check; (5) reject functions with nothing to print or return; (6) drop duplicates by content hash; (7) the round-trip test. Table I shows where the 67,662 source items went. We re-ran the static stages (1–6) deterministically over the same corpus range to get exact counts per stage; the LLM and execution stages appear as one aggregate row, since replaying them would need new LLM calls.

TABLE I
Acceptance funnel over 67,662 source items. Stages apply in order; each row counts items dropped at that stage.

Stage	Dropped	% of total
Empty after comment stripping	2	0.0
Duplicate of an accepted item	2,784	4.1
Quality bounds	24,183	35.7
File I/O / GUI / toolbox	12,971	19.2
Self-containedness check	18,174	26.9
No observable output	8	0.0
Round-trip test failed	4,244	6.3
Accepted	5,296	7.8

Read the funnel from top to bottom and the design logic appears. The cheap static stages run first and do most of the work: only 9,540 items reach the expensive round trip, and 55.5% of those pass. The round trip still earns its keep at the end: it rejects 4,244 descriptions that could not rebuild the program's behaviour—exactly the bad annotations that overlap-based screening would have let through. One detail: all 735 flat scripts that reached the round trip failed it, so the released dataset contains only functions.

A note on what the test does and does not prove. Each pair runs on one set of inputs. Matching outputs on one input set is strong evidence of faithfulness, not a proof of equivalence on all inputs. The round trip is a high-precision filter, not a theorem. A random sample of 50 accepted pairs is kept, with all artefacts, for manual audit.

E. What Is in the Dataset

The result is 5,296 validated pairs, all MATLAB functions, drawn from the MNBVC MATLAB corpus.[2] Functions average 24.7 lines (median 18, range 5–100). Descriptions average 314.6 words (median 272, range 47–1,052). Each sample has seven files: the cleaned source, the description, the rebuilt code, the harness, both outputs, and the source-corpus index. The train/test split is 90/10 (4,764/532), assigned by content hash, so a sample never switches sides between releases.

Compared to overlap- or judge-based screening, our criterion is strictly harder: a description must contain enough information to rebuild the program's behaviour, not just to resemble a reference. The price is yield—7.8%—and a bias toward small, self-contained, numeric code (Section IX).

[2]semran1/yulan-code-MNBVC-matlab on HuggingFace.

TABLE II
Semantic nodes for a small example function.

Depth	Type	Text
0	function_decl	function y = test(x)
1	if_condition	if x > 0
2	assignment	x = x + 1
2	assignment	y = x * 2

IV. Case Study: Can a Few Vectors Replace the Code?

With trustworthy data in hand, we returned to the idea that started the project. Code is a tree, and a tokenizer flattens it. Could we hand the decoder the tree directly—and could a compact structural representation even replace the code text? The task is translation $f_\theta : C \to S$ from code to pseudocode, generated token by token:

$$P(s_1, \ldots, s_T) = \prod_{t=1}^{T} P(s_t \mid \mathbf{z},\, s_{<t}), \tag{1}$$

where $\mathbf{z}$ is the representation of the code. Four variants differ only in $\mathbf{z}$; the decoder is always Qwen3-4B-Instruct [?] (36 layers, hidden size 2560).

A. From Tree to Vector, One Forced Step at a Time

Handing the decoder the tree dictates a chain of steps, each answering the question the previous one raises. To use the hierarchy we must first recover it: an ANTLR4 parser turns the source into an abstract syntax tree (AST), from which we extract semantic nodes—function declarations, conditions, loops, assignments—each with its nesting depth (Table II). The nodes are still raw text, so each must be given a meaning: frozen CodeBERT encodes every node into a 768-dimensional vector. The decoder cannot read CodeBERT's space, so a trained projector bridges the two: $f_{\text{proj}} : \mathbb{R}^{768} \to \mathbb{R}^{2560}$. What remains is deciding how many of these vectors reach the decoder—and that choice defines the variants.

B. The Four Variants

Text-only (baseline). The decoder reads the raw code through its own tokenizer (up to 512 tokens), then the instruction prompt. Plain fine-tuning; no encoder. This is what the vector variants must beat.

Tree. The decoder cannot consume a whole tree, so a recursive neural network (RvNN) folds it: children merge into their parent, level by level, until the root yields one vector $\mathbf{z}_{\text{tree}} \in \mathbb{R}^{2560}$ that summarizes the entire program. The decoder input is that single vector, then the prompt.

ViT. Borrowing the Vision Transformer recipe from computer vision, node vectors are grouped four at a time into "patches," each projected to one decoder vector. A typical function becomes about 5 vectors.

Combined. The tree vector, then the patches, then the prompt—structure and sequence together.

Note what the three vector variants share: the code text is gone. The decoder sees only projected vectors and a 12-token instruction. The whole program arrives as 1 vector (Tree), about 5 (ViT), or about 6 (Combined). This extreme compression is the hypothesis under test (RQ3), not an accident.

TABLE III
Quality on the 100 test items shared by all variants. Every vector-variant deficit vs. Text-only is significant at $p < 10^{-4}$. On the larger evaluation sets: Tree 0.173 ROUGE-L (n=532), ViT 0.176 (n=532), Combined 0.182 (n=180).

Variant	R-1	R-2	R-L	BLEU	chrF
Text-only	0.570	0.307	0.416	0.148	0.459
Combined	0.288	0.070	0.182	0.036	0.263
ViT	0.269	0.067	0.176	0.033	0.252
Tree	0.263	0.064	0.173	0.033	0.248

C. Training

Training has two stages. Stage 1 fine-tunes the decoder on plain code-to-pseudocode pairs (5 epochs, lr $2{\times}10^{-4}$; the last 18 of 36 layers, the final norm, and the output head unfrozen). Stage 1 is both the Text-only baseline and the starting point for stage 2. Stage 2 trains each vector variant's projector (lr $3{\times}10^{-4}$, 10 epochs), with the same 18 decoder layers unfrozen at a small learning rate ($1{\times}10^{-5}$). CodeBERT stays frozen. Both stages minimize cross-entropy on the target tokens. All models train on the 4,764 training pairs.

Two details of this setup looked harmless at training time and turn out to drive the whole result. First: with 18 layers plus the head unfrozen, 2,205,713,408 parameters are trainable in stage 2—against a code representation of at most six vectors. Second: the training targets carry no end-of-sequence token and are cut at 512 tokens, so the model is never shown where a description should stop. Keep both in mind for Section VII.

V. Experimental Setup

We evaluate on the held-out 532-sample test split, which no training stage ever saw. Generation is greedy, capped at 512 tokens. We report ROUGE-1/2/L, BLEU, and chrF, plus efficiency numbers measured in the same runs. Because some evaluation runs were interrupted, the variants cover different amounts of the test split (Tree and ViT: all 532; Combined: 180; Text-only: 100). All comparisons in Table III therefore use the 100 items covered by all four variants. Where a variant has a larger evaluation set, its full-set mean differs from its common-100 mean by at most 0.006 ROUGE-L. Significance uses a paired bootstrap with 10,000 resamples.

VI. Results

We expected the structural variants to help. They did not—and the way they failed is the finding.

RQ2: plain fine-tuning works well. The Text-only decoder reaches 0.416 ROUGE-L and 0.570 ROUGE-1 (Table III, Fig. 2). MATLAB code is full of English words—function and variable names—and the validated targets

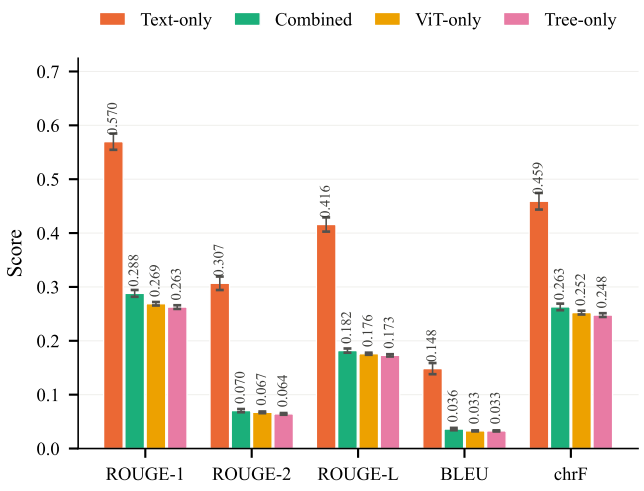

Fig. 2. Quality by variant (bars show SEM). The text baseline more than doubles every metric relative to the three vector variants.

are precise, so a fine-tuned decoder produces largely faithful step-by-step descriptions.

RQ3: every vector variant collapses. All three land in a narrow band, 0.17–0.18 ROUGE-L—less than half the baseline. The differences among the three are tiny (about 0.009) with overlapping confidence intervals, so we do not rank them, and we do not claim that combining the two encoders helps. The real question is why all three fail in the same way—and the answer, in Section VII, is that they are not describing the input badly. They are not describing the input at all.

Efficiency. The compression works exactly as designed: 13–18 input vectors instead of 242 text tokens (Fig. 3), and a smaller KV cache (46 vs. 66 MB). We attach no value to this: compression that loses the signal is not efficiency. Peak VRAM (7.7–8.7 GB) is dominated by the decoder everywhere; generation speed is about 19 tokens/s for all variants. One number, though, is a clue to what follows: every vector variant hit the 512-token generation cap on 100% of samples (Text-only: 96%).

VII. Why the Vector Variants Fail

Low scores alone could mean many things: noisy descriptions, wrong style, too much text. So we read the generations. What we found was stranger than low quality: the three vector models produce almost the same output no matter which program they are shown. We call this template collapse, and this section traces it to its causes.

A. First, Establish the Symptom

Three measurements, all computable from the released evaluation files:

- They never name the function. The Text-only model names the function it is describing in 80% of samples. All three vector variants: 0%. They do output function names—wrong ones, like get_size or getdata.

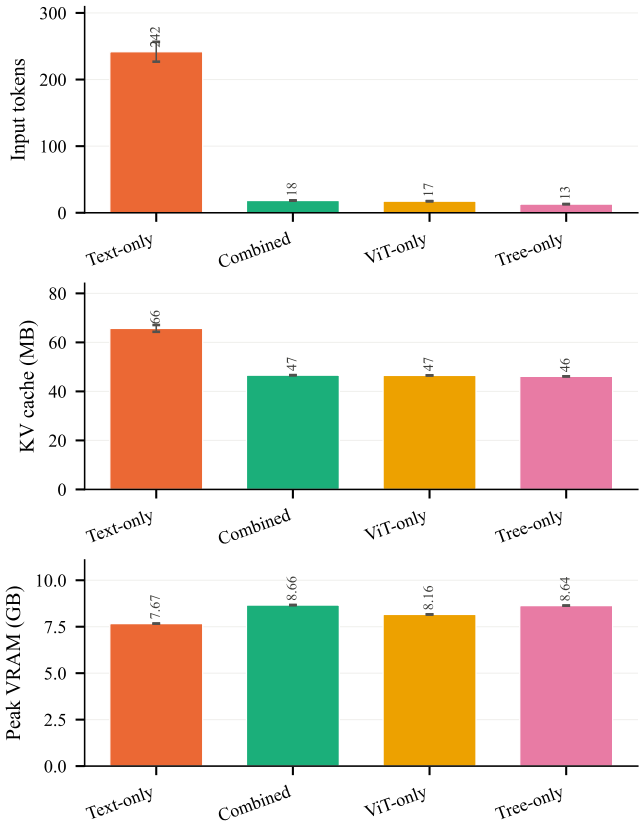

Fig. 3. Efficiency by variant: input width, KV cache, peak VRAM. The vector variants compress the input about 13×—and lose it.

- They repeat themselves. 29% of Tree outputs start with the same eight words (“1. Define a function named get_size that takes”). ViT: 19%. Combined: 9%. The Text-only model’s most common opening appears once.
- Wrong references barely hurt them. Scoring the vector models against randomly shuffled references costs them almost nothing. Most of their 0.17 is what any fluent, numbered, MATLAB-flavoured step list scores against these references.

Table IV shows what this looks like on one input. The function builds a rotation matrix. Text-only describes it correctly. The three vector variants confidently describe three different programs that do not exist.

```
function R = rotx(t, deg)
 if nargin > 1 && strcmp(deg, 'deg')
     t = t*pi/180;
end
ct = cos(t); st = sin(t);
R = [1 0 0; 0 ct -st; 0 st ct];
```

B. Then, Trace the Causes

Why would a trained model ignore its own input? Working backwards from the symptom, we found four reasons, one deep and three that compound it.

TABLE IV
Output openings for the rotx function above. The three vector variants describe programs that do not exist.

Variant	R-L	Output (beginning)
Reference	—	1. Define a function named rotx that takes two inputs: an angle t and an optional parameter deg. 2. Check if the number of input arguments is greater than 1 AND if the value of deg equals the string 'deg'…
Text-only	0.60	1. Define a function named rotx that takes two inputs: an angle t and an optional string/flag deg. 2. Check if the number of input arguments is greater than 1 AND the value of deg is equal to the string 'deg'…
Tree	0.15	1. Define a function named m2mat that takes a single input parameter x and returns a value y. 2. Check if the number of input arguments provided to the function is less than 2…
ViT	0.19	1. Define a function named getAxes that takes one input argument, nargin. 2. Check if the number of input arguments is less than 2…
Combined	0.14	1. Define a function named sqr that takes a single input argument x and returns a value y. 2. Check if the number of input arguments provided is less than 1…

C0 — The decoder was not frozen. Recall the two details flagged in Section IV-C. Stage 2 left 18 decoder layers plus the output head trainable: 2,205,713,408 parameters, on 4,764 samples, with the code arriving as at most six vectors. Under cross-entropy, the model had two ways to lower its loss. It could learn to decode six opaque projected vectors—hard. Or it could use two billion parameters to memorize what MATLAB pseudocode typically says—every description starts "1. Define a function named…", argument checks come next, loops after that—easy. It chose easy. The degeneracies measured above are exactly the fingerprint of a memorized template. This is why systems like Frozen [?], BLIP-2 [?], and prefix tuning [?] freeze the decoder: freezing removes the shortcut. Ours was open, and the model took it.

C1 — The prefix is too narrow to fight back. For the prefix to beat memorization, it would need to carry enough information to lower the loss more than memorizing does. One to six vectors cannot. CodeBERT already crushes each statement into one vector; patching then merges four statements into one. Even a decoder that wanted to use the prefix gets about 5 vectors of signal next to 12 tokens of fixed prompt.

C2 — The instruction points at nothing. The decoder input is: soft vectors, then the prompt "Convert the following MATLAB code…", then the target. The prompt was kept unchanged from the text variant, so it promises code that follows—and nothing follows. One more reason to ignore the input.

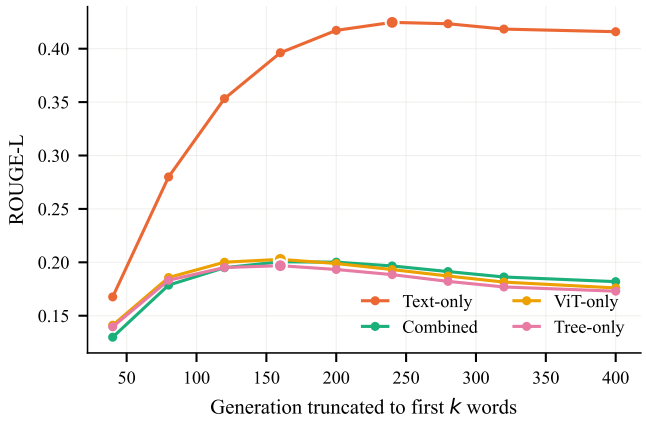

Fig. 4. ROUGE-L when each generation is cut to its first k words. Every model peaks at 160–240 words but generates 282–328 on average, because no end-of-sequence token was in the training targets. The enlarged marker is each model's peak.

C3 — The model is never taught to stop. The targets have no end-of-sequence token and are cut at 512 tokens, while references average about 305 words. So generation runs to the cap almost always—the clue we flagged in the efficiency results. Fig. 4 shows the cost: each model's score peaks at 160–240 words, then declines as it keeps talking, with actual generations averaging 282–328 words. Raising the cap would lower scores. This bug hits all four variants equally, so it cannot explain the gap between them; we report it because it lowers every score and is cheap to fix.

C. Finally, What Would Fix It

The causes imply their own ordering of remedies. Fixing C3 (add the EOS token) helps every variant but does not touch the collapse. Fixing C2 (write a prompt that refers to the vectors) is free but also not enough. The changes that matter are C0 and C1 together: freeze the decoder, so the training signal must flow through the projector; and widen the prefix—one vector per statement instead of per patch, or better, structure added to the code text rather than instead of it. And before any retraining, one cheap check: compute training loss with the correct prefix and with a prefix taken from a different sample. If the difference is near zero, conditioning is dead. We suggest this as the first experiment for any follow-up.

VIII. Discussion

For dataset builders. The round trip is a practical, automatic acceptance test for LLM-written code annotations. It needs only an interpreter, and it guarantees something overlap metrics cannot: an accepted description provably contains enough information to rebuild the program's behaviour. Two lessons travel beyond MATLAB. First,

validate the validator: without the self-containedness check, 18,174 broken functions would have nearly tripled the round-trip pool, and their "passes" would have meant nothing. Second, expect low yield: at 7.8%, this is a filter for corpus building, not a patch for an existing benchmark.

For model builders. Our negative result shows a failure that is easy to create and easy to miss. A collapsed model does not score zero—it scores 0.17, which looks like "weak but working." Aggregate metrics did not expose it. Three cheap probes did: does the output name the function? do outputs share openings? does shuffling references change the score? All three run on stored generations in minutes. We recommend them as standard checks for any prefix-conditioned model.

What the dataset enables next. Every pair ships with its harness and outputs. So the dataset supports execution-grounded model evaluation: score a generated description by whether code rebuilt from it reproduces the original outputs. This removes reference-overlap metrics—and their known bias toward LLM writing style—entirely. Our released pipeline includes this evaluator. Applying it to strong text-conditioned models is the natural next study.

IX. Threats to Validity

One annotator model. All descriptions come from one LLM. Execution validation bounds how wrong they can be, but not how uniform their style is, and overlap metrics partly reward matching that style.

One input set per pair. The round trip checks behaviour on one generated input set. That is strong evidence of faithfulness, not proof of equivalence on all inputs.

Selection bias. The funnel keeps small, self-contained, deterministic, numeric functions. I/O-heavy, toolbox-dependent, and long MATLAB code is under-represented; flat scripts are absent.

One training run. Each variant was trained once, with one hyperparameter setting, and the vector variants share the stage-1 initialization. The diagnosis describes this configuration. We separately verified that evaluation loaded all trained encoder weights, ruling out a checkpoint-loading artefact.

Depressed absolute scores. The missing-EOS bug (C3) lowers every absolute score. Relative comparisons are unaffected, since it applies to all variants equally.

X. Conclusion

This paper told two connected stories. The first is about trust: we needed thousands of code descriptions, an LLM wrote them, and we made each one prove itself—by letting an independent model rebuild the code from the description alone and checking, by execution, that the rebuilt program behaves like the original. A static self-containedness check keeps that proof honest, and it mattered far more than expected, removing a quarter of the corpus. The result is 5,296 pairs whose annotations are backed by runnable evidence.

The second story is about a belief that did not survive contact with the first. We expected compressed structural vectors to help a decoder understand code, perhaps even to replace the code text. Instead, all three vector designs collapsed into writing the same generic pseudocode for every input, while plain text fine-tuning worked well. The explanation is simple in hindsight—two billion trainable decoder parameters against a six-vector prefix make memorization cheaper than understanding—and its signature is measurable with three probes that any prefix-conditioned project can run in minutes. The two stories need each other: only because the supervision was verified can the failure be pinned on the model, and not on the data.

Acknowledgment